

Write-Up — TP Réseaux

TCP & UDP

Yanni Delattre Balcer

Date : 04/05/2026

Machine : Ubuntu 22.04 desktop

Passerelle : RT-Box2 (Nat/Interne)

Hyperviseur : Oracle VirtualBox

VM	Rôle	IP
RT-Box2 (Nat/Interne)	Passerelle	eth0: 10.0.2.15 / eth1: 192.31.25.1
Ubuntu 22.04 desktop	Machine de travail	192.31.25.x

PARTIE TCP — Transmission Control Protocol

Q1. Lancer Wireshark sur l'interface lo avec filtre TCP

Wireshark doit être lancé avec les droits root pour accéder à l'interface loopback :

```
sudo wireshark &
```

Interface sélectionnée : **lo (Loopback)** — Filtre de capture : **tcp**

■ La première tentative sans sudo n'affichait pas l'interface lo. Il faut obligatoirement sudo pour avoir accès à toutes les interfaces.

Q2. Lancer un serveur netcat sur le port 8000

```
nc -l 8000
```

Le serveur écoute sur la boucle locale (127.0.0.1) sur le port 8000.

Q3. Lancer un client netcat

```
nc localhost 8000
```

■ Important : toujours lancer le serveur AVANT le client. Lors du TP, le client a été lancé avant le serveur, ce qui a provoqué un RST/ACK immédiat (port fermé). Il a fallu relancer proprement dans le bon ordre.

Q4. Vérifier le port ouvert avec ss

```
ss -n | grep 8000
```

On utilise **ss** (paquet **IPROUTE2**) et non **netstat** (paquet NET-TOOLS, obsolète). De même, on utilise **ip** à la place de ifconfig, arp, route...

Q5. Échange de messages et fermeture

Messages tapés dans les terminaux :

```
Bonjour  
Salut  
Ca va  
Oui super
```

Fermeture de la connexion avec **Ctrl+D**.

Q6. Follow TCP Stream

Clic droit sur une trame → **Follow > TCP Stream**

Le flux affiche tous les messages échangés : Bonjour, Salut, Ca va, Oui super — **35 bytes** au total, 4 paquets serveur, 0 tour.

Q7. Flow Graph TCP

Statistiques > Graphique des flux — Type de Flux : TCP Flows — Cocher "Limiter au Filtre d'Affichage"

Temps (s)	De → Vers	Flags	Info
83.871	8000 → 68788	SYN	Seq=0 — Client initie
83.871	58788 → 8000	SYN, ACK	Seq=0 Ack=1 — Serveur répond
83.871	8000 → 68788	ACK	Seq=1 Ack=1 — Connexion établie
92.995	58788 → 8000	PSH, ACK	Seq=1 Ack=9 — "Bonjour\n"
92.995	8000 → 68788	ACK	Ack=9
165.163	58788 → 8000	PSH, ACK	Seq=9 Ack=15 — "Salut\n"
165.163	8000 → 68788	ACK	Ack=20
165.165	58788 → 8000	PSH, ACK	Seq=20 Ack=26 — "Ca va\n"
165.165	8000 → 68788	ACK	Ack=26
166.023	58788 → 8000	PSH, ACK	Seq=26 Ack=36 — "Oui super\n"
166.023	8000 → 68788	ACK	Ack=36

Q8. Vérification des numéros de séquence et d'acquittement

Relation à vérifier : **Seq + Len = Ack**

Données	Seq	Len (octets)	Ack attendu	Vérifié
"Bonjour\n"	1	8	9	✓
"Salut\n"	9	6	15	✓
"Ca va\n"	15	6	21	✓
"Oui super\n"	21	10	31	✓

Q9. Champ Taille de Fenêtre (Win) > 65535

Le champ Win est codé sur **2 octets** → valeur max théorique = **65 535 octets**. Pourtant Wireshark affiche des valeurs bien plus grandes.

Raison : L'option TCP **Window Scale** (RFC 1323), négociée pendant le 3-way handshake dans les options TCP du segment SYN :

TCP Option - Window scale: 7 (multiply by 128)

Vraie fenetre = $65536 \times 2^7 = 8\,388\,608$ octets

Dans Wireshark : développer une trame SYN → **TCP Options > Window scale**.

Q10. Identification du 3-way handshake

Trame	Flags	Seq	Ack	Rôle
3	SYN	0	—	Client initie la connexion
4	SYN, ACK	0	1	Serveur accepte et synchronise
5	ACK	1	1	Connexion établie

■ Par défaut, Wireshark affiche des numéros RELATIFS (premier Seq = 0). Pour voir les vrais ISN : Edit > Preferences > Protocols > TCP > décocher 'Relative sequence numbers'. Le SYN consomme 1 numéro de

séquence même sans données.

Q11. Identification de la fermeture de connexion

Après **Ctrl+D**, fermeture en **4 temps** (half-close) :

Étape	Flags	Qui	Rôle
1	FIN, ACK	Initiateur (Ctrl+D)	Demande de fermeture
2	ACK	Récepteur	Acquitte le FIN
3	FIN, ACK	Récepteur	Ferme son sens
4	ACK	Initiateur	Acquitte le FIN

■ C'est une fermeture half-close : chaque sens est fermé indépendamment. Le FIN consomme également 1 numéro de séquence.

Q12. Plusieurs onglets / connexions SSH — se mélangent-elles ?

Non, les connexions ne se mélangent jamais.

Une connexion TCP est identifiée par un **tuplet à 4 éléments** (socket) :

(IP source, Port source, IP destination, Port destination)

Chaque onglet ou session SSH utilise un **port source différent** attribué aléatoirement par l'OS (ex: 54231, 54232...). Même si l'IP et le port destination sont identiques (ex: port 22 pour SSH), le port source différent rend chaque connexion unique.

Q13. Connexion vers un port fermé

nc localhost 9999

Observation Wireshark :

Trame	Flags	Description
1	SYN	Client tente de se connecter
2	RST, ACK	Machine refuse (port fermé) — trame en ROUGE dans Wireshark

■ C'est exactement ce qui s'est passé au début du TP quand le serveur nc n'était pas encore lancé (2 premières trames en rouge dans Wireshark). Le RST signifie 'port inconnu / connexion refusée'. Pas de handshake, refus immédiat.

Q14. Connexion coupée — ré-émission TCP

Simulation : SSH vers une machine puis coupure réseau.

Ce qui se passe : TCP ne reçoit plus d'ACK → entre en ré-émission avec l'algorithme **Binary Exponential Backoff**.

Tentative	Délai approximatif
1ère	~1 s
2ème	~2 s
3ème	~4 s

4ème	~8 s
5ème	~16 s
...	Double à chaque fois

Le timer n'est pas fixe — il double à chaque ré-émission (Backoff exponentiel). Cela évite de saturer un réseau congestionné tout en persistant pour des coupures transitoires.

Durée totale : TCP insiste **~15 minutes** sous Linux (paramètre `tcp_retries2` = 15 tentatives par défaut).

Si la machine se reconnecte :

— Avant l'abandon → la connexion reprend normalement.

— Après l'abandon → la machine distante répond **RST** (plus d'état en mémoire) → connexion définitivement perdue.

PARTIE UDP — User Datagram Protocol

Q15. Analyse Wireshark d'un échange UDP

Terminal 1 — Serveur UDP :

```
nc -u -l 9000
```

Terminal 2 — Client UDP :

```
nc -u localhost 9000
```

Filtre Wireshark : **udp**

Capture Wireshark : seulement **2 datagrammes** envoyés directement sans aucun mécanisme de suivi :

Trame	Source → Destination	Longueur	Observation
1	127.0.0.1:49278 → 127.0.0.1:9000	Len=8	Données envoyées directement
2	127.0.0.1:49278 → 127.0.0.1:9000	Len=6	Données envoyées directement

Comparaison TCP vs UDP

Caractéristique	TCP	UDP
Établissement connexion	3-way handshake (SYN/SYN-ACK/ACK)	Aucun — envoi direct
Envoi des données	Après handshake uniquement	Immédiat
Accusé de réception	ACK systématique	Aucun
Numéros de séquence	Oui — renumérotation	Non
Fermeture connexion	FIN/ACK en 4 temps	Aucune
Fiabilité	Fiable (ré-émission)	Non fiable
Vitesse	Plus lent	Plus rapide
Utilisation typique	HTTP, SSH, FTP, SMTP	DNS, DHCP, VoIP, Streaming, Jeux

■ Les datagrammes UDP sont envoyés directement, sans suivi. Si un datagramme est perdu, UDP ne le sait pas et ne réémet pas. C'est à l'application de gérer si nécessaire (ex : DNS refait une requête, RTP ignore la perte).

POUR ALLER PLUS LOIN — Scans nmap

Commandes utilisées (avec sudo obligatoire)

Première tentative sans sudo :

```
nmap -sS localhost
```

Résultat : **"You requested a scan type which requires root privileges. QUITTING!"**

Correction — relance avec sudo :

```
sudo nmap -sS localhost
sudo nmap -sT localhost
sudo nmap -sA localhost
sudo nmap -sF localhost
sudo nmap -sN localhost
sudo nmap -sX localhost
```

Résultat commun à tous les scans

```
PORT STATE SERVICE
631/tcp open ipp
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.05 seconds
```

Un seul port ouvert détecté : **631/tcp — IPP** (Internet Printing Protocol).

Analyse des scans

Scan	Flags envoyés	Port ouvert	Port fermé	Caractéristique
-sS SYN	SYN	SYN/ACK puis RST	RST	Discret, handshake incomplet
-sT Connect	SYN complet	Handshake complet	RST	DéTECTABLE, connexion complète
-sA ACK	ACK	RST	RST	Détecte les règles pare-feu
-sF FIN	FIN	Rien	RST	Discret
-sN Null	Aucun flag	Rien	RST	Très discret
-sX Xmas	FIN+PSH+URG	Rien	RST	Tous les flags allumés

Dans Wireshark, on observe les trames **FIN**, **PSH**, **URG** (Xmas scan) avec les réponses **RST**, **ACK** pour les ports fermés. Plus de **14 000 paquets** générés au total par l'ensemble des scans.

■ Ces scans ne sont autorisés QUE sur ses propres machines. Scanner une machine tierce sans autorisation est ILLEGAL.

Conclusion

Ce TP a permis de mettre en évidence les différences fondamentales entre TCP et UDP :

TCP est un protocole orienté connexion, fiable, avec :

- Établissement via le 3-way handshake (SYN / SYN-ACK / ACK)
- Numérotation des octets (Seq/Ack)
- Accusés de réception systématiques
- Contrôle de flux (Window Scale)
- Ré-émission en cas de perte (Backoff exponentiel)
- Fermeture propre en 4 temps (FIN/ACK)

UDP est un protocole sans connexion, rapide, sans garantie de livraison, adapté aux applications temps réel : DNS, VoIP, streaming, jeux en ligne.

Les scans nmap illustrent comment les flags TCP peuvent être manipulés pour sonder des machines à des fins de reconnaissance réseau.